Dynamic Retrieval Augmented Code Generation

Dunhan Jiang*
CBD
Carnegie Mellon University
dunhanj@andrew.cmu.edu

Zian Pan*
LTI
Carnegie Mellon University
zianp@andrew.cmu.edu

Zhilan Wang*
LTI
Carnegie Mellon University
nicolewa@andrew.cmu.edu

1 Assignment Notes

This project satisfies criterion (1) by introducing new techniques for an existing task. Specifically, we introduce a novel multi-component reranking framework and dynamic retrieval system for code generation. Section 4.1 describes the work we completed previously in Assignment 3. The remaining sections are our new contributions developed for Assignment 4. For detailed methodology regarding the multicomponent reranking framework, please refer to Section 4.2. The implementation of the dynamic code generator is described in Section 4.3. The combination of these components creates a comprehensive system that significantly improves existing code retrieval methods.

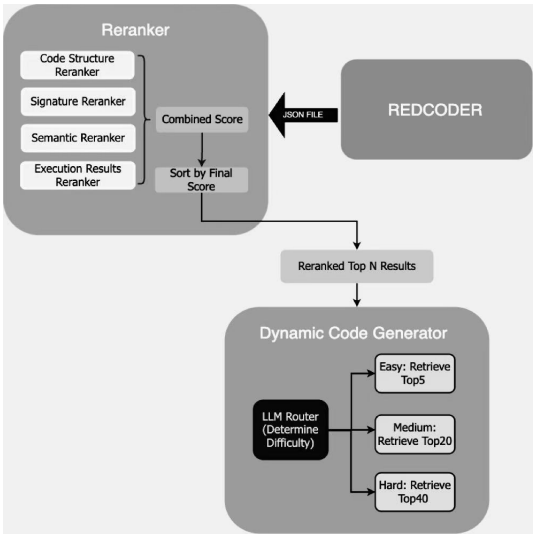

Figure 1: Overall architecture of our code generation system. The framework consists of two main components: (1) a multi-component reranker, and (2) a dynamic code generator

*Equal contribution.

2 Introduction

Retrieval-Augmented Generation (RAG) systems (Lewis et al., 2021) combine a generative model with an external knowledge source to produce better outputs. For code generation, RAG(Lewis et al., 2021) helps solve problems that require knowledge beyond what a model knows. For example, when writing code with unfamiliar libraries or APIs, developers often need to consult documentation or study previous code examples. Although the current development of the LLM has shown promising results, they are still struggling to understand the context, syntax, and user intent of the code. In this paper, we discovered the potential issues with the current RAG-based code generation models and applied a possible solution to resolve those problems. Building upon the RECODER (Parvez et al., 2021), we developed a customized Reranker and a Dynamic Code Generation system. Our code is made public on Github: https://github.com/zianpan/fancy_retriever.git

3 Motivation

Raw retrieval systems, including REDCODER (Parvez et al., 2021), simply fetch code based on basic similarity metrics without evaluating code quality or functional correctness. The extracted snippets often contain syntax errors, missing imports, or incorrect function signatures that directly conflict with the developer requirements. Without proper reranking, developers will get irrelevant results that match keywords but do not solve the actual problem. Therefore, we developed a comprehensive reranking framework that evaluates output from multiple dimensions - structural patterns, semantic meaning, function signatures, and execution success - to surface truly usable code. In addition, we believe that fixed retrieval methods blindly fetch the same number of code examples regardless of query complexity or actual needs. For example,

processing 40+ code snippets for simple tasks not only wastes computational resources but also introduces unnecessary latency. More critically, excessive irrelevant examples can pollute the generation process with confusing patterns that actually degrade code quality. As a result, our team decide to create a Dynamic Code Generator that adapts the number and type of retrieved examples based on the characteristics of the query, optimizes resource usage, reduces processing time, and maintains generation quality by providing the right amount of context for each specific programming task.

4 Methodology

4.1 REDCODER Reproduction

First part of our pipeline is to reproduce the SCODE-R retriever as discussed in the REDCODER (Parvez et al., 2021) paper. We carefully follow the methodology presented in the paper by employing a DPR architecture (Karpukhin et al., 2020) consisting of two distinct encoders: one for natural language summaries and another for source code. The process is smooth since the paper release checkpoints for python code retriever for the text-to-code task. Furthermore, we utilize pre-trained GraphCodeBERT (Guo et al., 2021) to encode the code-description pairs in our implementation. The model was downloaded from HuggingFace (Wolf et al., 2020).

Regarding the second part of our pipeline, the generative component, our primary objective was to closely replicate the functionality described in the original SCODE-G architecture (Parvez et al., 2021). The initial implementation leveraged a PLBART model (Ahmad et al., 2021) fine-tuned specifically for generating code snippets from natural language prompts enhanced by examples of retrieved code. However, due to the unavailability of the original PLBART (Ahmad et al., 2021) checkpoint and the extensive computational resources required for re-finetuning PLBART (Ahmad et al., 2021) on our target dataset, we developed a novel retrieval-augmented approach to address these limitations effectively. In our revised pipeline, each input text prompt is systematically augmented by concatenating a varying number of top-ranked retrieved code examples obtained from our reconstructed SCODE-R (Parvez et al., 2021) retrieval module. To ensure clarity and proper contextual separation between these retrieved examples and the original user input, we introduced a distinctive delimiter token. This carefully structured input format equips the subsequent generation models with extensive, contextually relevant cues, significantly enhancing their ability to accurately synthesize the target code. The output JSON file from REDCODER (Parvez et al., 2021) is then fed into our Multi-Component Reranking Framework.

4.2 Multi-Component Reranking Framework

We began by loading the output JSON file that contains coding-related pairs. Each pair includes a programming question submitted by a user and a corresponding answer produced by SCODE-G (Parvez et al., 2021). With the data loader, we preprocessed the datasets to keep a consistent question-answer-context pair list output. With this cleaned dataset, we performed the feature extraction. Specifically, we recorded the function signatures (such as function name and parameter list), control-flow counts (such as the frequency of "if-else"), imported modules, number of function calls, and line counts (the lines of the script).

Then, we started the re-ranking process by analyzing the code snippets from multiple aspects to determine the code score. To facilitate this analysis, we employed an Abstract Syntax Tree(AST) (Jones, 2024) component extractor. The extractor walks through the AST tree to extract essential elements, including function definitions, parameters, type annotations, docstrings, and control structures. Here is a detailed explanation about the rerankers:

Code Structure Reranker The reranker helps count the "control-flow" structures for both the reference answer and the retrieved code snippets. Specifically, it helps count the number of "if", "else", "for", "while", "try", "except", and the number of function calls. Then, it will compare these numbers between the reference answer and the retrieved code snippets. For each feature, the reranker calculates a normalized similarity score using the formula:

$$\text{similarity} = 1.0 - \frac{|\text{actual} - \text{expected}|}{\max(\text{actual}, \text{expected})}$$

Each control flow structure has a weight of 0.5, and function calls have a weight of 0.5. The final structure score is computed as $\frac{\text{score}}{\text{total weights}}$ where total weight is the sum of all feature weights. If the numbers are close to each other, the score for the code snippet will be high. The reranker also compares the length of the code, giving preference to snippets with line counts similar to the expected answer.

Signature Reranker We used the query intent extracted by the data processor to help with this reranker. Specifically, we measured the degree of overlap between the user query intent and the retrieved code snippets. The reranker uses regular expressions to extract function signatures. During the process, we awarded the code snippets with credits for exact matches (weight 2.0), substring matches (weight 1.6), and token-level matches (weight 1.0). For token similarity, we split function names and compute the intersection over union. Parameter matching considers both name similarity and position.

Semantic Reranker This reranker first extracts the multi-work key phrases and domain terms from the user query. Similar to previous Rerankers, it also uses regular expressions to extract verb phrases that often describe actions. Domain terms are extracted by checking against predefined dictionaries for common programming domains (file operations, string processing, math, web, database, error handling). Then, it calculated the semantic score by counting the presence of the key phrases and domain terms in the extracted snippet. The reranker also analyzes docstring quality by computing term overlap between query and docstring, giving parameter documentation higher weight than general term overlap.

Execution Results Reranker We used this to generate simple test inputs for the functions based on the signatures. The reranker first normalizes the code formatting using regular expressions to fix whitespace and handle single-line function definitions. It extracts function information using AST (Jones, 2024), falling back to regex parsing when AST fails. Test inputs are generated based on parameter names and default values, with domain-specific cases. The execution happens in a sandboxed environment using temporary Python files and subprocess calls with a timeout. The reranker creates a test script that imports the function and runs it with test inputs, capturing results in JSON format. The execution score is calculated as $\frac{\text{successful\_tests}}{\text{total\_tests}}$, with a default weight of 5.0 to emphasize working code.

Combined Reranker After calculating the scores for all rerankers, we combine these scores together with dynamically adjusted weights based on query characteristics. For example, when processing a file operation query like "function to read and parse a CSV file", the system would add additional weights to the SignatureReranker weight since file_io operations benefit from precise function signatures. Similarly, for a complex algorithm query such as "implement quicksort with partition function", the CodeStructureReranker weight will increase to emphasize proper control flow structure. The system also adjusts the weights based on the complexity level. High-complexity queries like "create a recursive binary tree traversal with error handling" receive a boost to the CodeStructureReranker weight to ensure proper structural matching. Moreover, query length affects weighting as well. Longer queries exceeding 50 words, such as detailed function specifications with multiple requirements, trigger an increase in SemanticReranker weight to better capture nuanced semantic requirements. Finally, we calculate the weighted sum of all scores from all rerankers to determine the final score for each code snippet. To ensure equal contribution from all rerankers, the final score is normalized using the formula:

$$\text{normalized\_score} = \frac{\text{score} - \text{min\_score}}{\text{max\_score} - \text{min\_score}}$$

Note: All the weights are determined through empirical testing and heuristic rules designed to optimize reranking performance for different types of programming queries.

4.3 Dynamic Code Generator

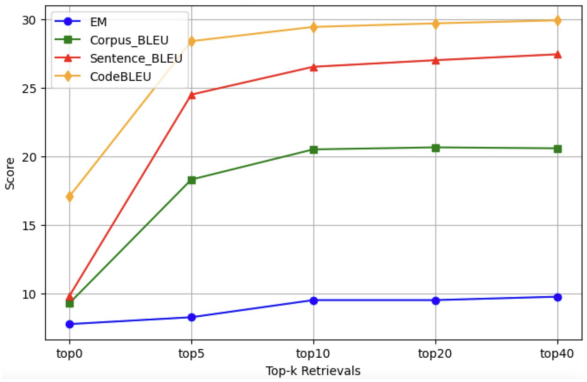

Figure 2: SCODE-G base code generator performance increases and becomes saturated with increasing numbers of retrieved code instances.

To further enhance our SCODE-G pipeline (Parvez et al., 2021), we integrated the Qwen2.5-14B-Instruct-1M(Yang et al., 2025) long-context transformer model, capitalizing on its ability to handle significantly longer context windows compared to baseline PLBART(Ahmad et al., 2021). This advancement enabled us to experiment effectively with incorporating a greater number of

retrieved code instances into the generation process. We hypothesized that providing the model with an increased number of retrieved instances would progressively enrich the contextual information available during generation, thereby improving the accuracy, relevance, and overall quality of the generated code.

The Qwen2.5-14B-Instruct-1M(Yang et al., 2025) model's superior capacity to process extensive inputs allows us to rigorously test and evaluate our hypothesis. By incrementally increasing the number of retrieved code snippets included in the input, we systematically assessed the correlation between the number of instances and the performance metrics such as exact match accuracy, BLEU(Papineni et al., 2002), and CodeBLEU (Ren et al., 2020) scores. Our empirical evaluations aimed to validate the hypothesis that an increase in retrieved context significantly contributes to improved code synthesis outcomes. Additionally, we think some text prompts for code generation will be relatively easier, while some other prompts will be harder. We thus utilize the Qwen2.5-14B-Instruct-1M model(Yang et al., 2025) to determine the difficulty level of a given text prompt, which is classified into easy, medium, or hard three levels. This step dynamically determines the number of retrieved code instances to be concatenated as inputs for subsequent code generation: easy problems will be allocated with the least amount of retrieved code instances (top 5), medium problems get top 20 code instances, while hard problems will get the largest amount of code instances (top 40).

5 Results

We evaluated our retrieval-augmented code generation framework across two benchmark datasets: CodeXGlue-CSNet(Lu et al., 2021) and CodeRAGBench-MBPP(Austin et al., 2021) As shown in Table 1 and Table 2, we compared the performance of the baseline retrievers, the generative models and our improved retrieval-augmented setup using multiple configurations.

For both datasets, we observe that simple retrieval methods such as SCODE-R and SCODE-R (Parvez et al., 2021) with Reranker perform modestly in isolation, with low exact match (EM) scores, but some signal in BLEU (Papineni et al., 2002) and CodeBLEU (Ren et al., 2020). Pure generative models such as PLBART (Ahmad et al., 2021) and Llama3.1-8B-Instruct(Touvron et al., 2023) offer slightly improved generation quality, but still fail to produce high-quality, task-relevant code. For example, on CodeXGlue-CSNet (Lu et al., 2021), PLBART (Ahmad et al., 2021) only achieved a BLEU (Papineni et al., 2002) score of 3.52 and a CodeBLEU (Ren et al., 2020) score of 10.39.

In contrast, our retrieval-augmented generation approach—particularly those using Llama3.1-8B-Instruct (Touvron et al., 2023) and Qwen2.5-14B-Instruct-1M (Yang et al., 2025) with reranked top-20 or top-40 retrieved code instances—consistently outperforms baselines. On CodeXGlue-CSNet(Lu et al., 2021), Qwen2.5-14B-Instruct-1M(Yang et al., 2025) with top-40 retrieval achieved an EM of 9.75, BLEU(Papineni et al., 2002) of 27.44, and CodeBLEU(Ren et al., 2020) of 29.92, while similar saturation was observed on CodeRAGBench-MBPP(Austin et al., 2021) with top-40 retrieval yielding 10.00 EM, 27.90 BLEU(Papineni et al., 2002), and 30.32 CodeBLEU(Ren et al., 2020).

These results suggest a clear trend of performance saturation as observed in Figure 2, where the benefit of additional retrieved examples plateaus beyond 20 or 40 instances. Nonetheless, the top-tier performance of our retrieval-augmented framework substantially exceeds the original REDCODER(Parvez et al., 2021) benchmark and demonstrates the strength of integrating dynamic reranking and long-context transformers in real-world code generation tasks. Moreover, to mitigate the saturating trend we observed previously and to enhance the efficiency of code generation, we introduced a dynamic code generation strategy using the same Qwen2.5(Yang et al., 2025) model to classify the given text prompt into easy, medium, or hard difficulty levels, each of them requesting top-5, top-20, and top-40 retrieved code instances, respectively, for code generation. As reported in Table 1 and Table 2, our LLM router enhanced dynamic retrieval augmented code generation system achieves comparable results as compared to the top-40 code instances augmented code generation on both datasets, while only consuming a quarter of the total compute on average.

6 Limitation

Our Retrieval Code Generation model proved to be very efficient and useful. It combines current traditional RAG models (Lewis et al., 2021) with our customized Reranker and Dynamic Code Gen-

Type	Method	EM	BLEU	CodeBLEU
Retriever	SCODE-R	0.00	19.50	20.12
	SCODE-R + Reranker	0.00	21.09	21.95
Generative	PLBART	0.00	3.52	10.39
	Llama3.1-8B-Instruct	0.00	8.62	14.95
	Qwen2.5-14B-Instruct-1M	1.25	10.53	17.08
Retrieval Augmented Code Generation using Reranker	Llama3.1-8B-Instruct (top-5)	7.50	22.97	26.92
	Llama3.1-8B-Instruct (top-10)	8.75	24.92	28.11
	Llama3.1-8B-Instruct (top-20)	8.75	25.36	28.66
	Llama3.1-8B-Instruct (top-40)	9.50	26.10	28.89
	Qwen2.5-14B-Instruct-1M (top-5)	8.25	24.50	28.40
	Qwen2.5-14B-Instruct-1M (top-10)	9.50	26.53	29.44
	Qwen2.5-14B-Instruct-1M (top-20)	9.50	27.01	29.70
	Qwen2.5-14B-Instruct-1M (top-40)	9.75	27.44	29.92
	LLM Router + Qwen2.5-14B-Instruct-1M	9.50	26.93	20.36

Table 1: Evaluation results on the CodeXGlue-CSNet across different code generation approaches. We compare retrieval-based (SCODE-R, SCODE-R + Reranker), pure generative (PLBART, Llama3.1-8B-Instruct, Qwen2.5-14B-Instruct-1M), and retrieval-augmented methods using Reranker. Metrics include Exact Match (EM), BLEU, and CodeBLEU (Ren et al., 2020). Our choice of LLM Router is also Qwen2.5-14B-Instruct-1M.

Type	Method	EM	BLEU	CodeBLEU
Retriever	SCODE-R	0.00	21.75	22.68
	SCODE-R + Reranker	0.00	23.55	20.12
Generative	PLBART	0.00	3.28	11.23
	Llama3.1-8B-Instruct	0.00	9.37	16.04
	Qwen2.5-14B-Instruct-1M	1.50	11.02	17.85
Retrieval Augmented Code Generation using Reranker	Llama3.1-8B-Instruct (top-5)	7.50	23.41	27.38
	Llama3.1-8B-Instruct (top-10)	9.00	25.30	28.56
	Llama3.1-8B-Instruct (top-20)	9.25	25.81	29.10
	Llama3.1-8B-Instruct (top-40)	9.25	26.55	29.42
	Qwen2.5-14B-Instruct-1M (top-5)	8.50	24.91	28.79
	Qwen2.5-14B-Instruct-1M (top-10)	9.75	26.91	29.81
	Qwen2.5-14B-Instruct-1M (top-20)	10.00	27.43	30.11
	Qwen2.5-14B-Instruct-1M (top-40)	10.00	27.90	30.32
	LLM Router + Qwen2.5-14B-Instruct-1M	9.75	27.35	20.89

Table 2: Evaluation results on CodeRagBench-MBPP (Similar to Table 1)

eration. Despite its great performance, it still has some limitations that might need to be improved in future research. The current model requires a significant amount of time to process the user queries. Specifically, it needs approximately 5-10 minutes to process dataset that contains 20 generated solutions for 100 coding questions. Moreover, the model might not be able to generate 100% correct test cases and answers, which might require human involvement during the generation process. Moreover, due to the limited budget, we did not train our Rerankers for better performance. Our large dataset would need a substantial processing time and computational resources for training the Rerankers.

7 Future Work

There are several approaches we could try to further improve the current model. For example, we could implement parallel processing to optimize the current Execution Results Reranker. Specifically, we could distribute different questions and answers to different nodes to enable parallel testing and answer generation. Also, we believe that, with more GPUs, we could fully fine-tune the current model to achieve the best performance. Distributed training would greatly improve the efficiency of the training process and allow us to incorporate even more training data. These possible works could address the current limitations and improve the current model's efficiency, accuracy, and resource management.

8 Conclusion

Overall, our work shows significant promise in enhancing the accuracy and practicality of code generation. Through a comprehensive review of prior work and our reproduction of the REDCODER framework (Parvez et al., 2021), we identified the strengths and limitations in current retrieval-based code generation methods. Our improvements to the REDCODER (Parvez et al., 2021) pipeline have demonstrated the potential benefits of dynamically adjusting the amount of retrieved code context based on query complexity. Moreover, our multi-component reranking framework addresses critical limitations in existing code retrieval systems by implementing a comprehensive evaluation pipeline

that considers structure, semantics, signatures, and execution viability.

References

Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. *Preprint*, arXiv:2103.06333.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *Preprint*, arXiv:2108.07732.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie LIU, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. Graphcode{bert}: Pre-training code representations with data flow. In *International Conference on Learning Representations*.

Joel Jones. 2024. Abstract syntax tree implementation idioms. PDF. Archived from the original on 21 July 2024.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Preprint*, arXiv:2005.11401.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *Preprint*, arXiv:2102.04664.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.

Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. *Preprint*, arXiv:2108.11601.

Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *Preprint*, arXiv:2009.10297.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Huggingface's transformers: State-of-the-art natural language processing. *Preprint*, arXiv:1910.03771.

An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, Junyang Lin, Kai Dang, Kexin Yang, Le Yu, Mei Li, Minmin Sun, Qin Zhu, Rui Men, Tao He, Weijia Xu, Wenbiao Yin, Wenyuan Yu, Xiafei Qiu, Xingzhang Ren, Xinlong Yang, Yong Li, Zhiying Xu, and Zipeng Zhang. 2025. Qwen2.5-1m technical report. *Preprint*, arXiv:2501.15383.